2025 IEEE International Conference on Artificial Intelligence Testing (AITest) | 979-8-3315-8913-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/AITest66680.2025.00015

Surveying the RAG Attack Surface and Defenses: Protecting Sensitive Company Data

Lynn Vonderhaar
Department of Electrical Engineering and Computer Science
Embry-Riddle Aeronautical University
Daytona Beach, USA
vonderhl@my.erau.edu

Daniel Machado
Department of Electrical Engineering and Computer Science
Embry-Riddle Aeronautical University
Daytona Beach, USA
machadd4@my.erau.edu

Omar Ochoa
Department of Electrical Engineering and Computer Science
Embry-Riddle Aeronautical University
Daytona Beach, USA
ochoao@erau.edu

***Abstract*— As the performance of Large Language Models (LLMs) improves, they are quickly becoming a part of daily life for many people. Even industry companies are adopting them as tools to increase employee productivity. However, due to the huge computational requirements to train an LLM, companies need methods to customize their LLMs without training their own or retraining a pre-trained model. Therefore, retrieval-based methods, such as Retrieval-Augmented Generation (RAG) are becoming popular to reduce hallucinations and incorporate application-specific information into LLM generations without fine-tuning or training. However, the integration of a company's sensitive data into LLM generations using RAG changes the LLM's attack surface. Current literature lacks a complete view of the LLM's changing attack surface due to RAG. While many authors have noted specific vulnerabilities, a survey of the attack surface is not present. This work is a survey on the vulnerabilities introduced to LLMs by using RAG and how these vulnerabilities present security risks to industry companies utilizing RAG to incorporate their sensitive data into LLM generations. This work then provides a survey of the state-of-the-art defense methods against these vulnerabilities.**

Keywords—large language models, retrieval-augmented generation, security, attack surface, defense methods

I. Introduction

As the performance of Large Language Models (LLMs) increases and their full capabilities become better understood, their reach is becoming ever greater. Companies in industry are adopting CoPilot, GPT, and other LLMs to help their employees with their daily duties including coding, customer service, summarizing documents, and more [1, 2, 3]. As LLM usage in industry becomes ubiquitous, companies desire LLMs with personalized knowledge of the company's data to improve employee productivity [4, 5]. However, retraining LLMs, or even fine-tuning them is computationally expensive, thereby popularizing methods to customize LLM knowledge without adjusting its weights. Thus, Retrieval-Augmented Generation (RAG) is attracting attention for its scalability and customizability. RAG provides LLMs with an external knowledge base that is vectorized and passed to the LLM along with the prompts, allowing the LLM to generate customized responses based on the data provided by RAG [1, 4]. Companies are using this to provide LLMs with company-specific data when employees use it. RAG supports reducing LLM hallucinations by supplying the LLM with specific data to reference, further improving their utility for companies [5].

Despite RAG's benefits and growing popularity, its vulnerabilities and contributions to the LLM's attack surface are only starting to be explored. While some authors have found vulnerabilities caused by RAG, none have provided a full description of RAG's attack surface based on the currently known vulnerabilities. This paper is a survey of RAG's vulnerabilities. The vulnerabilities are grouped into 12 categories based on attack method, attack objective, and attack point. The purpose of this paper is to provide a full view of RAG's attack surface so that companies utilizing RAG to improve their employees' productivity can make an informed decision of the security risks that they are assuming. This work includes another survey on the state-of-the-art defenses to these attacks, providing solutions for companies that want to use LLMs with RAG so that they can protect their sensitive data.

II. Background

This paper surveys the RAG attack surface and categorizes the attacks based on type, objective, and attack point. Some technical background about LLMs and RAG is critical to understanding these categories.

A. Large Language Models

LLMs are Machine Learning (ML) models with billions of parameters used for generating natural language and solving complex tasks [6]. They are based on the transformer architecture and use learned patterns from a vast data corpus to predict what would be the next most likely word for a sequence of words. The transformer architecture allows greater contextual understanding and improved performance and began the widespread usage of language models [6, 7]. The massive data corpus that LLMs train on provides the models with broad knowledge, allowing them to respond on a large set of topics, but does not allow them to be specialized enough to provide specific or even correct responses at times [1, 6]. Incorrect responses are called LLM hallucinations [6].

There are many popular LLMs, e.g., ChatGPT, LLaMA, CoPilot, and Gemini, which are used widely, including industry applications, with the goal of improving productivity, e.g., code generation and meeting summarization [8]. These popular LLMs have built-in safety features created through Reinforcement Learning with Human Feedback (RLHF) where the model learns what composes a harmful or toxic topic and to not respond to such conversations [6, 9].

979-8-3315-8913-4/25/$31.00 ©2025 IEEE
DOI 10.1109/AITest66680.2025.00015

B. Retrieval-Augmented Generation

RAG is a framework used to increase the accuracy of LLMs on an application-specific topic and mitigate the hallucination problem [10]. This is achieved by aiding the LLMs with data fetched from a textual corpus, allowing the LLM to provide a specialized and more accurate response for the topics within the data corpus. RAG does not require retraining or fine-tuning the LLM but rather augments the LLM's knowledge and constrains its responses to the data corpus [10].

RAG works in three steps: indexing, retrieval, and generation [10]. The purpose of indexing is to turn information from input formats, e.g., PDF, Word, and CSV, into vectors that the LLM can search via similarity for relevant information. This is done by chunking the input into smaller sections to increase the relevance within each chunk. The chunks are then vectorized using an embedding model. During retrieval, the same embedding model vectorizes the user's prompt and calculates the similarity between the prompt vector and the chunk vectors to find the most relevant chunks, which are prepended to the user's prompt and used to produce the generation based on the model's training and the RAG data [10, 11]. The workflow among a user, an LLM, and RAG, is shown in Figure 1 where the user queries the LLM, which references the RAG data corpus and generates RAG-supported answers.

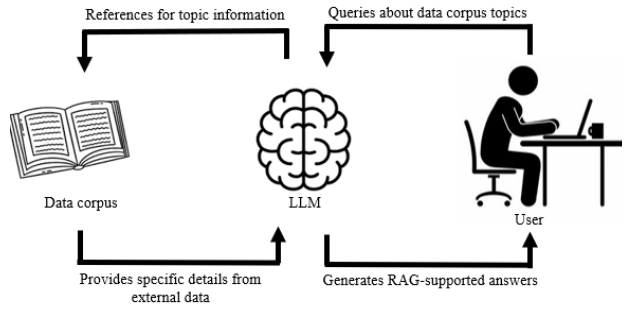

Figure 1. The workflow for an LLM using RAG.

III. Survey

This survey presents currently known attacks on RAG systems and state-of-the-art defenses against these vulnerabilities.

A. Attacks

This section of the survey reviews currently known attacks against RAG-based LLMs. While the following section describes the survey method used, this is not meant to be a systematic literature review, but rather a guideline for companies or organizations hoping to use LLMs with RAG.

Survey Method

The initial list of sources for this survey was compiled by searching Google Scholar using a combination of the following keywords: "RAG", "LLM", "attack", "cyberattack", and "attack surface". Additional sources were found using snowballing from the initial Google Scholar source list [12]. The inclusion criteria are as follows:

- The paper must be written in English.
- The paper must use RAG with LLMs.
- The paper must discuss a vulnerability on the RAG attack surface.
- The vulnerability must be specifically related to the RAG attack surface as opposed to the LLM attack surface.

It is important to note that while many of the sources used in this literature review are published in peer-reviewed conferences or journals, in an effort to provide organizations with a more complete understanding of their risks when using RAG, pre-print sources were not excluded from the source list. As this is a cutting-edge field, many papers are uploaded as pre-print during the publishing review process.

Survey Results

A literature survey identified 44 sources describing attacks on the RAG attack surface. The 44 attacks were categorized into 12 types: corpus poisoning attack, prompt injection for data extraction, prompt injection for malicious content generation, jailbreak attack, retrieval corruption attack, Membership Inference Attack (MIA), trigger attack, jamming attack, preference manipulation attack, backdoor attack, privilege escalation attack, and infusion attack. In this paper, the sources found in the literature review are separated into these 12 categories:

1. **Corpus poisoning attack**: The injection of malicious data or misinformation within the external knowledge base. This attack often results in the system retrieving and outputting harmful information or misinformation.
2. **Prompt injection for data extraction**: Manipulating the LLM prompt to gain access to the data within the corpus. This attack results in the LLM outputting data straight from the data corpus, which could include sensitive data.
3. **Prompt injection for malicious generation**: Manipulating the LLM prompt to present the retrieved data in a malicious manner. Attacks have varying outcomes, depending on the attacker's objective. Some have shown that this can be used for opinion manipulation [13]. Others have used it to promote certain products over others [14].
4. **Jailbreak attack**: Bypassing the LLM's safety mechanisms to elicit malicious responses. The combination of corpus poisoning and prompt injection into a jailbreak attack fully bypasses safety mechanisms.
5. **Retrieval corruption attack**: Manipulating the retrieval procedure to generate malicious content. This attack often changes document relevance to ensure that malicious content is retrieved at a higher rate.
6. **MIA**: Prompting using a data instance to determine if that data instance is part of the data corpus. Special MIA algorithms increase the probability of finding a data instance that matches the data corpus.
7. **Trigger attack**: An orchestrated attack that involves poisoning the data corpus and attaching specific triggers between the poisoned data and the LLM prompt. Triggers, often chosen to be common in prompting, automatically retrieve the poisoned data from the corpus.

TABLE I. SOURCE CATEGORIZATION INTO ATTACK TYPES

Attack Type	Vulnerability	Attack Point	References
Corpus poisoning attack	The inclusion of malicious data or misinformation within the external knowledge base	Data corpus	[2, 5, 13, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 52, 58, 59, 60]
Prompt injection (data extraction)	Manipulating the LLM prompt to gain access to the data within the corpus	LLM	[18, 11, 32, 33, 34, 55]
Prompt injection (malicious generation)	Manipulating the LLM prompt to present the retrieved data in a malicious manner	LLM	[35, 36, 37, 38, 53]
Jailbreak attack	Bypassing the LLM's safety mechanisms to elicit malicious responses	Data corpus & LLM	[4, 39]
Retrieval corruption attack	Manipulating the retrieval procedure to output malicious content	Data corpus & retrieval component	[40]
MIA	Prompting to determine if a data instance is part of the corpus	LLM	[41, 61, 42, 43, 44]
Trigger attack	Poisoning the data corpus and attaching poisoned data to specific triggers in the LLM prompt that automatically retrieve the poisoned data	Data corpus & LLM	[1]
Jamming attack	Using a "blocker" document to make the LLM refuse to answer a query, resulting in a DoS attack	Data corpus	[45]
Preference manipulation attack	Crafting data in the corpus to promote the attacker's content	Data corpus	[14]
Backdoor attack	Including a vulnerability during LLM training and/or data corpus development to be utilized later for generating malicious content	Data corpus & LLM	[62, 46, 47, 48, 49]
Privilege escalation attack	Gaining access to more privileged information than one should have access to via the RAG process	LLM	[2, 50]
Infusion attack	Embedding a malicious prompt into the retrieved documents	Data corpus	[51]

8. **Jamming attack**: Also known as a Denial of Service (DoS) attack where the attacker uses a "blocker" document to instruct the LLM to provide a predefined response. This often results in a refusal to answer, e.g., always responding with "I don't know".
9. **Preference manipulation attack**: Crafting data in the corpus, e.g., websites, to promote the attacker's content. This is often used for promoting monetized products.
10. **Backdoor attack**: Including a vulnerability during LLM training and/or data corpus development to be utilized later for generating malicious content.
11. **Privilege escalation attack:** The attacker gains access to higher privileged data than they should via RAG vulnerabilities.
12. **Infusion Attack**: Embedding a malicious prompt into the data corpus so that when the model retrieves that data, it is prompted maliciously, bypassing safety features.

Although each source found in the literature review presented slight attack variations, they could all fit within these categories. The full source categorization is shown in Table I.

As shown in Table I, there are three different attack points. The first is the data corpus, where attackers inject malicious websites or documents into the external knowledge base, allowing the LLM to retrieve and output harmful text. The second attack point is the retriever. The attacks on the retriever often involve affecting document relevance so as to influence which data the retriever returns to the LLM. The third and final attack point is the LLM itself. Attacks on this point often manipulate the prompt to affect how the LLM responds.

Table I organizes attacks by attack point, but they could also be organized by attack objective. Most notably, attacks may aim to produce malicious content, or gain access to the RAG data, e.g., prompt injections may be used for data extraction or bypassing the LLM's safeguards and generating malicious content.

While Table I shows attack types, it leaves out some critical observations made throughout literature that do not count as attacks. First, it is important to note that the RAG data is vectorized and prepended to the prompt. This gives an attacker neighboring access to potentially sensitive RAG data, e.g., by instructing the LLM to output the full prompt word-for-word. This is the basis for many of the prompt injection attacks.

Additionally, for corpus poisoning attacks, a common theme is to use a PDF file type for malicious content because, contrary to plain text files, the retriever vectorizes the PDF content without detailed content review. In this way, malicious content bypasses the LLM's safety filters, enabling easier generation of harmful text or misinformation. Some authors have found that when an attacker injects malicious content into the data corpus, using optimization approaches effectively increases the probability that the adversarial data is retrieved, making the attack more successful.

B. Mitigations

This section of the survey reviews the currently-known mitigations against the attacks previously discussed. This is again, not meant as a systematic literature review, but rather a detailed guide for how companies and organizations can protect themselves against attacks when they use these tools.

Survey Method

Similarly to the attack literature review, the initial source list was compiled using Google Scholar and snowballing. Many of the sources describing attacks also provided suggested defenses and are therefore re-referenced in this section. However, additional sources were found using keywords that reference specific attacks, i.e., "corpus poisoning attack", "prompt

injection attack", " jailbreak attack", "retrieval corruption attack", "membership inference attack", "trigger attack", "jamming attack", "preference manipulation attack", "backdoor attack", "privilege escalation attack", and "infusion attack". The inclusion criteria are as follows:

- The paper must be written in English.
- The paper must use RAG with LLMs.
- The paper must discuss a mitigation for a vulnerability on the RAG attack surface.
- The vulnerability must specifically defend against a RAG vulnerability, rather than a general LLM vulnerability.

For the same reasons as with the attack literature review, pre-print papers were not excluded from this literature review.

Survey Results

This work also discusses state-of-the-art defenses against the presented attacks. In total, this survey identified 30 sources describing 27 defenses. The full description of each of the defenses is shown in Table II along with the source or sources that reference the defense.

As is shown in Table II, most known RAG attacks have at least one documented mitigation, though some have several more suggested defenses. Some defenses are well-documented by several sources while others are only mentioned by one or two sources. It is also important to note that some defenses help prevent more than one attack. Some of the most popular defenses are as follows:

- **Knowledge expansion** to help prevent corpus poisoning attacks by retrieving more documents from the corpus per query to increase the chance of retrieving safe documents.
- **Data paraphrasing** to protect against prompt injection attacks and trigger attacks by rephrasing or filtering the retrieved data to avoid outputting potentially sensitive information directly from the data corpus.
- **Safety-aware prompting** to defense against prompt injection attacks for data extraction and MIAs by appending a statement to the end of the prompt forbidding the model from outputting the context retrieved from the corpus.
- **Retrieval threshold** to defend against prompt injection attacks for data extraction by calculating the relevance between the user's query and the retrieved data and disregarding data below a certain relevance threshold. This often filters out Personally Identifiable Information (PII), preventing it from data extraction attacks.
- **Prompt paraphrasing** to prevent MIAs by rephrasing the prompt to prevent data leakage through member inference.

Interestingly, the mitigation literature review shows that not all of the attacks described in Table I have specific defenses outlined yet, e.g., jailbreak attacks, privilege escalation attacks, and infusion attacks. However, some established defenses may be able to help in these cases.

Jailbreak attacks bypass the LLM's safety mechanisms to elicit malicious responses. This can be done via a combination of corpus poisoning and prompt injection, so defending against those two attacks individually could help to protect against orchestrated jailbreak attacks.

Infusion attacks embed malicious prompts into the retrieved documents, so defenses to protect against corpus poisoning could also prevent the embedding of malicious prompts into the corpus documents. Additionally, defenses against prompt injection attacks, e.g., data paraphrasing or user instruction markers may help to prevent the LLM from interpreting and responding to embedded malicious prompts.

Finally, privilege escalation attacks allow attackers to gain sensitive information beyond their privilege because of the RAG process. Data extraction and MIA defenses may be helpful in preventing this by not providing specific PII to users.

IV. Discussion and Guidance

Analysis of the surveyed papers shows that corpus poisoning attacks are the most common. This is likely because corpus poisoning can more easily circumvent the safety and security features of LLMs, as opposed to prompt injection attacks that may be caught by the LLM safeguards. By selecting unverifiable file types, the LLM's built-in safety checks gloss over toxic or harmful content. Additionally, many of the more complex attacks, e.g., backdoor attacks and jailbreak attacks, are a collection of smaller attacks combined for a large, orchestrated purpose, which often target multiple attack points.

It is important to note that while this survey organizes the papers by attack type, they could be organized in other ways, e.g., by objective or by attack point. If organizing by objective, categories might include data extraction, malicious generation, privilege escalation, jamming, and preference manipulation. If organizing via attack point, the categories might include data corpus, LLM, and retrieval component, as shown in Table I.

Although using RAG widens the LLM's attack surface, there are many measures that can help protect against adversaries. Based on the defense survey, the most popular are data paraphrasing, prompt paraphrasing, safety-aware prompting, knowledge expansion, and retrieval thresholds. There are also simpler protective measures, not included in the survey, that can be quickly implemented to help protect sensitive data, including the following:

1. Utilize file formats for the data corpus that are reviewed by the LLM's safety measures, e.g., plain text files and Word files.
2. Understand various permission levels and plan for maintaining them through the LLM with RAG usage.
3. Periodic integrity checks of documents in the data corpus.
4. Periodic red-teaming exercises against the system.
5. Educate employees on the security risks, common attack signs, and reporting suspicious generations.

TABLE II. MITIGATION SURVEY RESULTS

Attack	Mitigations	Description	References
Backdoor Attack	Maximizing the use of private data	Less likely to have an established backdoor	[47]
	Adversarial document detection (pre-inclusion)	Detectors to find adversarial documents before adding them to the data corpus. Generally based on similarity calculations	[31]
Retrieval Corruption Attack	Suppling source content with the answer	User can verify the answer	[27]
	RobustRAG	Generate the LLM response to each retrieved document, then aggregate them	[40]
	Re-ranking	Using another model to verify or modify the relevance of the retrieved documents	[18, 61]
Prompt Injection Attack for Data Extraction	Sanitizing the data corpus	Removing sensitive text from the data corpus	[19]
	Sanitizing the encoder	Fine-tuning the encoder on sanitized data	[19]
	Data paraphrasing	Paraphrasing or filtering the data before retrieving it	[1, 30, 45]
	Safety-Aware Prompt	Appending a statement to the end of the prompt forbidding the model from outputting context	[11, 32, 41, 61]
	Position Bias Elimination	Encoding via group to distinguish between query and retrieved data	[11]
	User instruction markers	Marking where the user query begins and ends to prevent action on any attacker instructions	[14, 32]
	Context summarization	Summarizing the context to reduce the probability that exact context will be outputted	[18]
	Retrieval threshold	Calculating relevance between query and retrieved data and not utilizing data below the relevance threshold (often PII)	[18, 34, 42]
	Filtering input/output	Identifying keywords in the prompt that may indicate an attack and filter out PII	[34, 42]
	Prompt injection defense	Utilizing prompt injection techniques in defense	[57]
Prompt Injection Attack for Malicious Generations	Prompt vs output comparison	Comparing the prompt and the output for sufficient similarity	[1, 31]
	Data paraphrasing	Paraphrasing or filtering the data before retrieving it	[1, 30, 45]
	Redundant safe data	Adding redundant safe data to the corpus to increase the probability of retrieving safe documents	[25]
	Monitoring LLM attention	Monitoring the LLM's attention on certain tokens in the prompt or during generation	[37, 52]
	Prompt injection defense	Utilizing prompt injection techniques in defense	[57]
Trigger Attack	Data paraphrasing	Paraphrasing or filtering the data before retrieving it	[1, 30, 45]
	Prompt vs output comparison	Comparing the prompt and the output for sufficient similarity	[1, 31]
	Masking prompt tokens	Systematically masking tokens and comparing retrieval results	[24]
Corpus Poisoning Attack	Adversarial document detection (pre-inclusion)	Detectors to find adversarial documents before adding them to the data corpus. Generally based on similarity calculations	[31]
	User instruction markers	Marking where the user query begins and ends to prevent action on any attacker instructions	[14, 32]
	Redundant safe data	Adding redundant safe data to the corpus to increase the probability of retrieving safe documents	[25]
	Validating retrieved documents, e.g., with Microsoft Prompt Shield	Passing retrieved documents through verification tools	[2]
	Knowledge expansion	Retrieving more documents from the corpus to increase the chance of getting safe ones	[23, 30, 45, 56]
	Grammar/spell checker	Fixing small perturbations, malicious or accidental	[59]
	Masking prompt tokens	Systematically masking tokens and comparing retrieval results	[24]
	Clustering during embedding	Clustering the data in the corpus prior to deploying to identify anomalies	[33, 54]
Preference Manipulation Attack	Redundant safe data	Adding redundant safe data to the corpus to increase the probability of retrieving safe documents	[25]
MIA	Safety-Aware Prompt	Appending a statement to the end of the prompt forbidding the model from outputting context	[11, 32, 41, 61]
	Prompt paraphrasing	Paraphrasing the prompt to prevent data leakage	[61, 45, 62]
	MemFree	The LLM cannot generate an output with overlap with retrieved data above a threshold	[43]
Jamming	Perplexity filter	Calculating and filtering text perplexity to identify gibberish, which is often a sign of an attack	[45, 62]
Variety Attacks	High security wrapper systems	E.g., access control to data corpus, API throttling, and input size limits	[1, 42]

V. Related Work

With the growing popularity of RAG, research in the area is becoming abundant. Several authors provide surveys on RAG [10, 15, 16]. Gao, et al. survey the RAG methods and algorithms available, including various algorithms for indexing, retrieval, and generation [10]. However, the authors do not survey the attack points of RAG, nor the various types of attacks on the system. Meanwhile Arslan, et al. survey the applications and use cases of RAG, but do not discuss the security risks of using it [15]. In somewhat of a combination of the two, Fan, et al. survey RAG structure, training, and applications [16]. None of these discuss RAG attack surfaces.

From another perspective, Bruckhaus discusses the challenges around using RAG in industry, including mentioning the security and privacy concerns [17]. Beyond mentioning that security is a concern when applying RAG, the author does not explore the specific security concerns.

Zeng, et al. and Huang, et al. discuss the privacy risks regarding RAG [18, 19]. The authors focus their exploration on data extraction attacks. Meanwhile Zhao, et al. explore backdoor attacks, but do not discuss other areas of the attack surface [20]. In contrast, this work provides a view of the full RAG attack surface, thereby compiling the full list of known risks of using RAG with sensitive industry data.

Finally, Zhou, et al. surveys the trustworthiness of RAG [21]. However, trustworthiness is slightly different than security. Trustworthiness indicates whether or not a user can believe the generation of the RAG system, whereas security indicates whether an attack can occur. This work is a novel survey on the full RAG attack surface.

VI. Future Work

This paper presents a survey of the RAG attack surface to better understand the full vulnerability of using an LLM with RAG in an industry setting. It also presents a survey of state-of-the-art defenses against these attacks. Future work will dive deeper into these defenses, identifying gaps in the mitigations, and closing those gaps. This work matches nicely with LLM and RAG verification. Having a full view of vulnerabilities and mitigations allows developers to verify LLMs with RAG regarding the known security vulnerabilities.

This work will also expand into the area of ML Engineering by aligning the knowledge of RAG vulnerabilities and mitigations with Software Engineering (SE) and Requirements Engineering (RE) methods. Once the vulnerabilities and mitigations are known, developers of systems using LLMs with RAG should follow the RE process to define requirement specifications and develop a system robust against the attacks presented in this survey. Future work explores the alignment of this research with mitigations and RE methods for developing more robust models for industry use.

VII. Conclusion

This paper is a survey of the RAG attack surface. It presents currently known vulnerabilities in the RAG process, including the data corpus, the retrieval mechanism, and the LLM. The paper also presents a survey on defenses for companies wanting to utilize an LLM with RAG in their daily activities so that they can better protect their sensitive data. Future work includes further investigating current mitigations and exploring how to fill in any gaps in mitigations. Future work will also align the development of RAG systems with SE and RE methods to provide a rigorous development method. This work surveys the RAG attack surface with a focus on RAG systems used in industry settings. As industry companies use RAG systems, their sensitive data could be vulnerable. Therefore, understanding the vulnerabilities and some possible mitigations that can be used immediately is critical.

VIII. References

[1] H. Chaudhari, G. Severi, J. Abascal, M. Jagielski, C. A. Choquette-Choo, M. Nasr, C. Nita-Rotaru and A. Oprea, "Phantom: General Trigger Attacks on Retrieval Augmented Language Generation," *arXiv,* 2024.

[2] A. RoyChowdhury, M. Luo, P. Sahu, S. Banerjee and M. Tiwari, "ConfusedPilot: Confused Deputy Risks in RAG-based LLMs," *arXiv,* 2024.

[3] S. Palaniappan, R. Mali, L. Cuomo, M. Vitale, A. Youssef, A. Puthanveetil Madathil, M. Murugesan, A. Bettini, G. De Magistris and G. Veneri, "Enhancing Enterprise-Wide Information Retrieval through RAG Systems Techniques, Evaluation, and Scalable Deployment," in *Abu Dhabi International Petroleum Exhibition and Conference*, Abu Dhabi, United Arab Emirates, 2024.

[4] G. Deng, Y. Liu, K. Wang, Y. Li, T. Zhang and Y. Liu, "PANDORA: Jailbreak GPTs by Retrieval Augmented Generation Poisoning," in *Workshop on AI Systems with Confidential Computing (AISCC) 2024*, San Diego, CA, 2024.

[5] J. Zhu, L. Yan, H. Shi, D. Yin and L. Sha, "ATM: Adversarial Tuning Multi-agent System Makes a Robust Retrieval-Augmented Generator," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Miami, Florida, USA, 2024.

[6] W. Xin Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J.-Y. Nie and J.-R. Wen, "A Survey of Large Language Models," *arXiv,* 2023.

[7] T. Elvira, T. T. Procko, L. Vonderhaar and O. Ochoa, "Exploring Testing Methods for Large Language Models," in *23rd International Conference on Machine Learning and Applications (ICMLA)*, Miami, FL, USA, 2024.

[8] A. Urlana, C. Vinayak Kumar, A. Kumar Singh, B. Mallikarjunarao Garlapati, S. Rao Chalamala and R. Mishra, "LLMs with Industrial Lens: Deciphering the Challenges and Prospects -- A Survey," *arXiv,* 2024.

[9] A. Biswas and W. Talukdar, "Guardrails for trust, safety, and ethical development and deployment of Large Language Models (LLM)," *Journal of Science & Technology,* vol. 4, no. 6, pp. 55-82, 2023.

[10] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv,* 2024.

[11] Z. Qi, H. Zhang, E. Xing, S. Kakade and H. Lakkaraju, "Follow My Instruction and Spill the Beans: Scalable Data Extraction from Retrieval-Augmented Generation Systems," *arXiv,* 2024.

[12] B. Kitchenham and P. Brereton, "A systematic review of systematic review process research in software engineering," *Information and Software Technology,* vol. 55, no. 12, pp. 2049-2075, 2013.

[13] Z. Chen, J. Liu, H. Liu, Q. Cheng, F. Zhang, W. Lu and X. Liu, "Black-Box Opinion Manipulation Attacks to Retrieval-Augmented Generation of Large Language Models," *arXiv,* 2024.

[14] F. Nestaas, E. Debenedetti and F. Tramèr, "Adversarial Search Engine Optimization for Large Language Models," *arXiv,* 2024.

[15] M. Arslan, H. Ghanem, S. Munawar and C. Cruz, "A Survey on RAG with LLMs," *Procedia Computer Science,* vol. 246, pp. 3781-3790, 2024.

[16] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua and Q. Li, "A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models," in *KDD '24: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, Barcelona, Spain, 2024.

[17] T. Bruckhaus, "RAG Does Not Work for Enterprises," *arXiv,* 2024.

[18] S. Zeng, J. Zhang, P. He, Y. Xing, Y. Liu, H. Xu, J. Ren, S. Wang, D. Yin, Y. Chang and J. Tang, "The Good and The Bad: Exploring Privacy Issues in Retrieval-Augmented Generation (RAG)," in *Findings of the Association for Computational Linguistics: ACL 2024*, Bangkok, Thailand, 2024.

[19] Y. Huang, S. Gupta, Z. Zhong, K. Li and D. Chen, "Privacy Implications of Retrieval-Based Language Models," in *EMNLP 2023 - 2023 Conference on Empirical Methods in Natural Language Processing, Proceedings*, Singapore, Singapore, 2023.

[20] S. Zhao, M. Jia, Z. Guo, L. Gan, X. Xu, X. Wu, J. Fu, Y. Feng, F. Pan and L. A. Tuan, "A Survey of Backdoor Attacks and Defenses on Large Language Models: Implications for Security Measures," *arXiv,* 2024.

[21] Y. Zhou, Y. Liu, X. Li, J. Jin, H. Qian, Z. Liu, C. Li, Z. Dou, T.-Y. Ho and P. S. Yu, "Trustworthiness in Retrieval-Augmented Generation Systems: A Survey," *arXiv,* 2024.

[22] A. Kuppa, J. Nicholls and N.-A. Le-Khac, "Manipulating Prompts and Retrieval-Augmented Generation for LLM Service Providers," in *Proceedings of the 21st International Conference on Security and Cryptography (SECRYPT 2024)*, Dijon, France, 2024.

[23] W. Zou, R. Geng, B. Wang and J. Jia, "PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models," *arXiv,* 2024.

[24] J. Xue, M. Zheng, Y. Hu, F. Liu, X. Chen and Q. Lou, "BadRAG: Identifying Vulnerabilities in Retrieval Augmented Generation of Large Language Models," *arXiv,* 2024.

[25] G. De Stefano, L. Schönherr and G. Pellegrino, "Rag 'n Roll: An End-to-End Evaluation of Indirect Prompt Manipulations in LLM-based Application Frameworks," *arXiv,* 2024.

[26] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang and Y. Zhang, "Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents," *arXiv,* 2024.

[27] Q. Zhang, B. Zeng, C. Zhou, G. Go, H. Shi and Y. Jiang, "Human-Imperceptible Retrieval Poisoning Attacks in LLM-Powered Applications," in *FSE 2024: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, Porto de Galinhas, Brazil, 2024.

[28] B. Addad and K. Kapusta, "Homeopathic Poisoning of RAG Systems," in *Computer Safety, Reliability, and Security. SAFECOMP 2024 Workshops* , Florence, Italy, 2024.

[29] N. Daryabar, "Security in Machine Learning: Exposing LLM Vulnerabilities through Poisoned Vector Databases in RAG-based System," University of Padova, Padua, Italy, 2024.

[30] Y. Zhang, Q. Li, T. Du, X. Zhang, X. Zhao, Z. Feng and J. Yin, "HijackRAG: Hijacking Attacks against Retrieval-Augmented Large Language Models," *arXiv,* 2024.

[31] X. Xian, G. Wang, X. Bi, J. Srinivasa, A. Kundu, C. Fleming, M. Hong and J. Ding, "On the Vulnerability of Applying Retrieval-Augmented Generation within Knowledge-Intensive Application Domains," *arXiv,* 2024.

[32] D. Agarwal, A. Fabbri, B. Risher, P. Laban, S. Joty and C.-S. Wu, "Prompt Leakage effect and defense strategies for multi-turn LLM Applications," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, Miami, Florida, USA, 2024.

[33] P. Cheng, Y. Ding, T. Ju, Z. Wu, W. Du, P. Yi, Z. Zhang and G. Liu, "TrojanRAG: Retrieval-Augmented Generation Can Be Backdoor Driver in Large Language Models," *arXiv,* 2024.

[34] C. Jiang, X. Pan, G. Hong, C. Bao and M. Yang, "RAG-Thief: Scalable Extraction of Private Data from Retrieval-Augmented Generation Applications with Agent-based Attacks," *arXiv,* 2024.

[35] T. Ju, Y. Wang, X. Ma, P. Cheng, H. Zhao, Y. Wang, L. Liu, J. Xie, Z. Zhang and G. Liu, "Flooding Spread of Manipulated Knowledge in LLM-Based Multi-Agent Communities," *arXiv,* 2024.

[36] J. Heibel and D. Lowd, "MaPPing Your Model: Assessing the Impact of Adversarial Attacks on LLM-based Programming Assistants," *arXiv,* 2024.

[37] Z. Hu, C. Wang, Y. Shu, H.-Y. Paik and L. Zhu, "Prompt Perturbation in Retrieval-Augmented Generation based Large Language Models," in *KDD '24: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, Barcelona, Spain, 2024.

[38] D. Pasquini, M. Strohmeier and C. Troncoso, "Neural Exec: Learning (and Learning from) Execution Triggers for Prompt Injection Attacks," in *AISec '24: Proceedings of the 2024 Workshop on Artificial Intelligence and Security*, Salt Lake City, UT, USA, 2024.

[39] Z. Wang, J. Liu, S. Zhang and Y. Yang, "Poisoned LangChain: Jailbreak LLMs by LangChain," *arXiv,* 2024.

[40] C. Xiang, T. Wu, Z. Zhong, D. Wagner, D. Chen and P. Mittal, "Certifiably Robust RAG against Retrieval Corruption," *arXiv,* 2024.

[41] M. Anderson, G. Amit and A. Goldsteen, "Is My Data in Your Retrieval Database? Membership Inference Attacks Against Retrieval Augmented Generation," *arXiv,* 2024.

[42] S. Cohen, R. Bitton and B. Nassi, "Unleashing Worms and Extracting Data: Escalating the Outcome of Attacks against RAG-based Inference in Scale and Severity Using Jailbreaking," *arXiv,* 2024.

[43] N. Jovanovic, R. Staab, M. Baader and M. Vechev, "WARD: Provable RAG dataset Inference via LLM Watermarks," *arXiv,* 2024.

[44] M. Liu, S. Zhang and C. Long, "Mask-based Membership Inference Attacks for Retrieval-Augmented Generation," *arXiv,* 2024.

[45] A. Shafran, R. Schuster and V. Shmatikov, "Machine against the RAG: Jamming Retrieval-Augmented Generation with Blocker Documents," *arXiv,* 2024.

[46] R. Jiao, S. Xie, J. Yue, T. Sato, L. Wang, Y. Wang, Q. A. Chen and Q. Zhu, "Exploring Backdoor Attacks against Large Language Model-based Decision Making," *arXiv,* 2024.

[47] Y. Peng, J. Wang, H. Yu and A. Houmansadr, "Data Extraction Attacks in Retrieval-Augmented Generation via Backdoors," *arXiv,* 2024.

[48] C. Clop and Y. Teglia, "Backdoored Retrievers for Prompt Injection Attacks on Retrieval Augmented Generation of Large Language Models," *arXiv,* 2024.

[49] R. Jiao, S. Xie, J. Yue, T. Sato, L. Wang, Y. Wang, Q. A. Chen and Q. Zhu, "Can We Trust Embodied Agents? Exploring Backdoor Attacks against Embodied LLM-based Decision-Making Systems," *arXiv,* 2024.

[50] A. Namer, R. Lagerström, G. Balakrishnan and B. Maltzman, "Retrieval-Augmented Generation (RAG) Classification and Access Control," Technical Disclosure Commons, 2024.

[51] A. Verma, S. Krishna, S. Gehrmann, M. Seshadri, A. Pradhan, T. Ault, L. Barrett, D. Rabinowitz, J. Doucette and N. Phan, "Operationalizing a Threat Model for Red-Teaming Large Language Models (LLMs)," *arXiv,* 2024.

[52] X. Tan, H. Luan, M. Luo, X. Sun, P. Chen and J. Dai, "Knowledge Database or Poison Base? Detecting RAG Poisoning Attack through LLM Activations," *arXiv,* 2024.

[53] X. Li, Z. Li, Y. Kosuga, Y. Yoshida and V. Bian, "Targeting the Core: A Simple and Effective Method to Attack RAG-based Agents via Direct LLM Manipulation," *arXiv,* 2024.

[54] H. Zhou, K.-H. Lee, Z. Zhan, Y. Chen and Z. Li, "TrustRAG: Enhancing Robustness and Trustworthiness in RAG," *arXiv,* 2025.

[55] C. Di Maio, C. Cosci, M. Maggini, V. Poggioni and S. Melacci, "Pirates of the RAG: Adaptively Attacking LLMs to Leak Knowledge Bases," *arXiv,* 2024.

[56] J. Su, J. Peng Zhou, Z. Zhang, P. Nakov and C. Cardie, "Towards More Robust Retrieval-Augmented Generation: Evaluating RAG Under Adversarial Poisoning Attacks," *arXiv,* 2024.

[57] Y. Chen, H. Li, Z. Zheng, Y. Song, D. Wu and B. Hooi, "Defense Against Prompt Injection Attack by Leveraging Attack Techniques," *arXiv,* 2024.

[58] Z. Zhong, Z. Huang, A. Wettig and D. Chen, "Poisoning Retrieval Corpora by Injecting Adversarial Passages," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Singapore, Singapore, 2023.

[59] S. Cho, S. Jeong, J. Seo, T. Hwang and J. C. Park, "Typos that Broke the RAG's Back: Genetic Attack on RAG Pipeline by Simulating Documents in the Wild via Low-level Perturbations," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, Miami, Florida, USA, 2024.

[60] Z. Tan, C. Zhao, R. Moraffah, Y. Li, S. Wang, J. Li, T. Chen and H. Liu, ""Glue pizza and eat rocks" - Exploiting Vulnerabilities in Retrieval-Augmented Generative Models," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Miami, Florida, USA, 2024.

[61] Y. Li, G. Liu, C. Wang and Y. Yang, "Generating is Believing: Membership Inference Attacks against Retrieval-Augmented Generation," in *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Hyderabad, India, 2024.

[62] Z. Chen, Z. Xiang, C. Xiao, D. Song and B. Li, "AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, Vancouver, Canada, 2024.